

MONITOR Z · L00

Qué es un demonio en z/OS y qué vamos a construir

Cuaderno de estudio · versión del 2026-10-05

Margen derecho y renglones libres para tus notas. Las láminas son las mismas de la carpeta **_visual**. Si una lección se corrige, se regenera este cuaderno con la fecha nueva.

L00 — Qué es un demonio en z/OS y qué vamos a construir

Objetivo: al terminar sabés explicar con tus palabras qué es un demonio, cómo se traduce cada idea de Unix al mundo z/OS, qué piezas tiene nuestro monitor y en qué punto está cada una. **Esta lección no tiene código.** Es el mapa para no perderse en las siguientes.

1. El demonio en Unix (la idea que ya conocés)

Un **demonio** (*daemon*) es un proceso que:

1. arranca solo, sin una terminal atada (con el sistema o con un comando);
2. vive mucho tiempo, en general mientras el sistema esté arriba;
3. pasa casi todo el tiempo **esperando** algo: un evento, un timer o una conexión;
4. cuando despierta hace un trabajo corto y vuelve a esperar;
5. obedece señales para que lo controlen desde afuera (`SIGTERM` para parar, `SIGHUP` para releer la configuración);
6. deja rastro en un log (`syslog`).

La clave es el punto 3: **un buen demonio no gasta CPU mientras espera.** No hace `while(1){ preguntar(); }`. Se duerme en `sleep`, `select/poll` o un semáforo, y el sistema operativo lo despierta.

🕒 Analogía: el sereno de un edificio

Llega solo a su turno (nadie lo trae de la mano), se queda toda la noche, pasa casi todo el tiempo **sentado esperando**, cuando suena el timbre hace algo corto y vuelve a sentarse, obedece al administrador ("andate", "cambiá la ronda") y anota todo en su cuaderno.

#	Un demonio...	El sereno...	Nuestro monitor ZTKMON...
1	arranca solo, sin terminal atada	llega solo a su turno	lo arranca el operador con <code>S ZTKMON</code> (versión STC)
2	vive mucho tiempo	se queda toda la noche	vive todo el cierre
3	pasa casi todo el tiempo esperando	está sentado, no camina sin parar	duerme con <code>STIMER</code> o <code>WAIT</code> : CPU ≈ 0
4	despierta, hace algo corto y vuelve a esperar	abre la puerta y se vuelve a sentar	pregunta a JES, manda los cambios y vuelve a dormir
5	obedece órdenes de afuera	"andate" / "cambiá la ronda"	<code>P ZTKMON</code> (parar) y <code>F ZTKMON,...</code> (ordenar)
6	deja rastro en un log	anota en su cuaderno	<code>SYSPRINT</code> y <code>WTO</code> a la consola

La más importante es la 3. Un buen demonio **no gasta CPU mientras espera**: el sistema operativo lo despierta.

Fuente: Lección L00, sección 1

Lámina L00-01_que_es_un_demonio

El bucle: mal demonio vs. buen demonio

❌ Mal demonio: pregunta sin parar

```
mientras siempre:
  preguntar si pasó algo
  (y vuelve a preguntar de inmediato)
```

- Usa la CPU al 100 % aunque no pase nada.
- Le roba tiempo a los jobs del cierre.
- Si encima le pregunta a JES, también lo carga a él.

✅ Buen demonio: duerme y despierta

```
mientras siempre:
  dormir hasta que me despierten
  o pasen N segundos
  ver qué cambió y avisar
```

- Dormido: **CPU ≈ 0**.
- Trabaja sólo un instante en cada vuelta.
- Avisa sólo los **cambios**, no todo de nuevo.

👶 Analogía: el niño en el auto vs. la alarma del celular

El mal demonio es el niño que pregunta "¿ya llegamos?" cada segundo: no avanza más rápido y cansa a todos. El buen demonio pone una **alarma** ("despertame en 5 minutos o cuando lleguemos") y se duerme.

Para dormir, en...	Unix	z/OS	Lo vemos en
Un tiempo fijo	<code>sleep(5)</code>	<code>STIMER WAIT,BINTVL=...</code>	L02
Hasta que me avisen	<code>sem_wait()</code>	<code>WAIT</code> sobre un <code>ECB</code> (otro hace <code>POST</code>)	L02
Lo primero que pase de varias cosas	<code>select()</code> / <code>poll()</code>	<code>WAIT ECBLIST=...</code>	L02, L08

Fuente: Lección L00, sección 1 · z/OS MVS Programming: Assembler Services Guide (STIMER, WAIT/POST)

Lámina L00-02_bucle_malo_vs_bueno

2. La misma idea en z/OS

Idea Unix	Equivalente z/OS	Lo vemos en
Proceso	Address space (espacio de direcciones), con sus tareas (TCB)	L07
Demonio que arranca con el sistema	STC (started task) : el operador o el arranque lo levanta con <code>S ZTKMON</code> . Su JCL sale de una PROCLIB y no ocupa un iniciador batch	L07
Proceso largo lanzado a mano	Job batch largo : ocupa un iniciador de su clase todo el tiempo que corre. Es lo que hace hoy ZTKMONS (v0)	L04
<code>sleep(5)</code>	<code>STIMER WAIT, BINTVL=...</code>	L02
Semáforo (<code>sem_wait / sem_post</code>)	ECB (<i>event control block</i> , una palabra de memoria) con <code>WAIT</code> y <code>POST</code>	L02
<code>select()</code> sobre varios descriptors	<code>WAIT ECBLIST=...</code> : despertar con el primer ECB que se postee	L02, L08
Señales (<code>SIGTERM</code> , <code>SIGHUP</code>)	Comandos del operador <code>P ZTKMON</code> (STOP) y <code>F ZTKMON, texto</code> (MODIFY), que llegan por el CIB (<code>EXTRACT ... FIELDS=COMM, QEDIT</code>)	L07
<code>syslog</code>	<code>WTO</code> a la consola / <code>SYSLOG</code> , y <code>SYSPRINT</code>	L01, L04
Preguntar al kernel por los procesos (<code>ps</code>)	Preguntar a JES2 por los jobs: SSI función 80 (<i>extended status</i>)	L03
Que el kernel avise (<code>inotify</code> , eventos)	ENF 70 : JES2 avisa cuando un job cambia de estado (SELECT, DESELECT, CHANGE, PURGE). Hace falta APF	L08
Ver quién tiene un archivo bloqueado (<code>lsof</code> , <code>flock</code>)	ENQ sobre el recurso <code>SYSDSN</code> y GQSCAN para listar dueños y esperas	L09
Correr como root	APF + estado supervisor / clave 0-7. Sólo lo piden ENF 70 y otros servicios autorizados	L08

 Fuente: los términos y servicios son los de *z/OS MVS Programming: Assembler Services Guide* (STIMER, WAIT/POST, ECB, EXTRACT, QEDIT, ENQ/GQSCAN), *z/OS MVS Using the*

Subsystem Interface (SSI 80) y z/OS MVS Programming: Authorized Assembler Services Reference (ENFREQ). La tabla de equivalencias es criterio propio, para orientarse.

De Unix a z/OS: tabla de equivalencias

L00 · Qué es un demonio en z/OS

Lo que conocés de Unix	En z/OS se llama	En nuestro monitor	Lección
Proceso	Address space (con sus tareas, TCB)	ZTKMON vive en el suyo	L07
Demonio que arranca con el sistema	STC (<i>started task</i>): S ZTKMON, JCL en una PROCLIB, sin iniciador batch	La versión final	L07
Programa largo lanzado a mano	Job batch largo : ocupa un iniciador todo el tiempo	ZTKMONS v0, hoy	L04
<code>sleep(5)</code>	STIMER WAIT	El "dormir" del bucle	L02
Semáforo	ECB + WAIT / POST	Despertar el bucle con eventos	L02
<code>select()</code>	WAIT ECBLIST=...	Timer + evento + orden del operador	L02, L08
Señales (kill , SIGTERM, SIGHUP)	P ZTKMON (STOP) y F ZTKMON,... (MODIFY), por el CIB	Parar u ordenar sin cancelar	L07
<code>syslog</code>	WTO (consola / SYSLOG) y SYSPRINT	El rastro del monitor	L01, L04
<code>ps</code> (listar procesos)	Preguntarle a JES2 por SSI 80	Lo que hace ZTKMONS en cada vuelta	L03
Que el kernel te avise (inotify)	ENF 70 : JES2 avisa SELECT, DESELECT, CHANGE y PURGE	La versión por eventos	L08
<code>lsof / flock</code>	ENQ sobre SYSDSN y GQSCAN	"¿Quién tiene el archivo?" (FDCICS02)	L09
Ser root	APF + estado supervisor / clave 0–7	Lo pide ENF 70	L08

Fuente: Equivalencias: criterio propio para orientarse · servicios según Assembler Services Guide, Using the Subsystem Interface y Authorized Assembler Services Reference (IBM)

Lámina L00-03_unix_vs_zos

📦 Analogía: la caja del supermercado

Los **iniciadores** son las cajas donde se atiende a los jobs. Un **job largo** es un cliente que se queda en la caja todo el día: mientras tanto, esa caja no atiende a nadie más. Un **STC** es un empleado de planta con su propio puesto: no ocupa ninguna caja, y el gerente (el operador) le puede hablar o mandarlo a su casa.

	Job batch largo (ZTKMONS v0, hoy)	STC (ZTKMON, versión final)
Cómo arranca	Se somete como cualquier job (SUBMIT)	El operador: S ZTKMON (o el arranque del sistema)
De dónde sale el JCL	Del miembro que sometés	De una PROC en la PROCLIB
Iniciador	Ocupa uno de su clase todo el tiempo	No usa iniciador batch: tiene su propio address space
Cómo se para	Termina solo (PARM de minutos) o hay que cancelarlo	P ZTKMON : para ordenado , cierra y avisa
Órdenes en vivo	No	F ZTKMON, ESTADO y otras
Quién lo autoriza	Nadie: alcanza con tu librería PPCE.ZTK	Sistemas: alta de la PROC, usuario (perfil STARTED) y APF si usa ENF 70
Para qué lo usamos	Aprender y validar (L04)	El monitor de verdad (L07, L08)

Fuente: Lección L00, sección 2

Lámina L00-04_job_largo_vs_stc

3. Por qué no alcanza con "preguntar cada tanto"

En el análisis de IDEA-005 (2026-09-30) se midió cuánto cuesta preguntarle a JES desde distintos lugares:

Vía	Costo por consulta	Evidencia
SDSF desde REXX (ISFEXEC ST)	9 a 11 s, porque recorre todo el spool	Cabecera de PPCE.ZTK.ASM(ZTKSSI) , DESA 2026-09-03
FTP-JES desde la PC	~0,3 s del lado de JES; el resto es el arranque de la CLI	3 corridas sobre PROD, 2026-09-30
SSI 80 desde Assembler	sin medir: lo mide la práctica del módulo 4	—

Conclusión de entonces: el costo lo ponen **la interfaz de JES y lanzar un job por consulta**, no el lenguaje. Por eso el diseño tiene tres niveles:

Nivel	Cómo funciona	Ventaja	Precio
-------	---------------	---------	--------

A. Polling desde la PC	FTP-JES cada 60 s	Nada en el host	Sin paso ni CC precisos; latencia de 60 s
B. Un job o STC que duerme	Bucle <code>STIMER WAIT</code> + SSI 80 cada N s; manda sólo los cambios	Hora de inicio exacta y CC; sin APF	Ocupa un iniciador (si es job) y consulta aunque no pase nada
C. Por eventos	Listen exit de ENF 70 , que hace POST del ECB; la tarea principal espera en WAIT	Aviso instantáneo y cero consultas en vano	APF + STC: trámite con sistemas

Construimos B primero (ZTKMONS) y C al final. B ya sirve, B valida que entendemos lo que dice JES, y C reusa casi todo B: cambia sólo *quién* despierta al bucle.

Cuánto cuesta preguntarle a JES (lo medido)

L00 · Qué es un demonio en z/OS

Vía	Costo por consulta	Evidencia
SDSF desde REXX (ISFEXEC ST)	9 a 11 s : recorre todo el spool	Cabecera de PPCE.ZTK.ASM(ZTKSSI) , DESA, 2026-09-03
FTP-JES desde la PC	~0,3 s de JES (3,42–3,46 s en total; 3,16 s son el arranque de la CLI de zowe)	3 corridas sobre PROD, 144 filas, 2026-09-30 ~08:50
SSI 80 desde Assembler (ZTKMONS)	sin medir	Lo mide la práctica del módulo 4 (CPU del JESMSG LG ÷ cantidad de vueltas)

Analogía: buscar un libro

SDSF desde REXX es como recorrer **toda la biblioteca** estante por estante para saber qué libros están prestados. SSI 80 es preguntarle directamente al **bibliotecario**, que tiene la lista.

Conclusión: el costo lo ponen **la interfaz de JES** y **lanzar un job por consulta**, no el lenguaje.

Fuente: IDEA-005, tabla "Medido hoy" (2026-09-30)

Lámina L00-06_costo_de_preguntar

📖 Analogía: saber quién entra al edificio

A. Llamás por teléfono a la portería cada minuto y preguntás "¿entró alguien?". **B.** Un vigilante adentro hace una ronda cada 5 segundos y anota sólo lo nuevo. **C.** Ponés un **timbre** en la puerta: suena justo cuando alguien entra, y nadie hace rondas en vano.

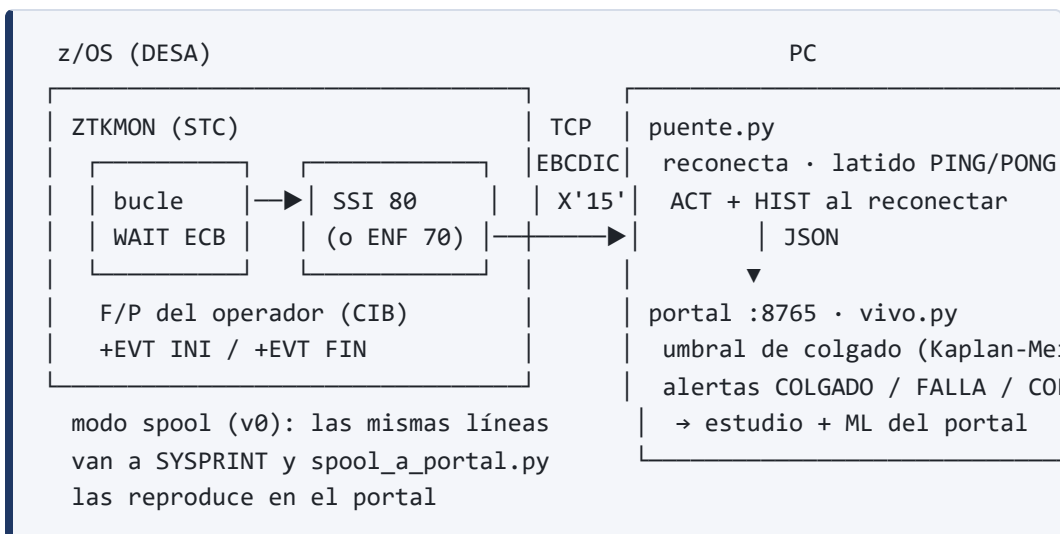
Nivel	Cómo funciona	Ventaja	Precio
A · Preguntar desde la PC	FTP a JES cada 60 s	No hay que tocar nada en el host	Sin paso ni CC precisos; hasta 60 s de atraso
B · Programa en el host que duerme y pregunta	Bucle: STIMER (dormir N s) → SSI 80 → mandar sólo los cambios	Hora de inicio exacta y CC. Sin APF	Pregunta aunque no pase nada; si es job, ocupa un iniciador
C · Por eventos	JES2 avisa por ENF 70 → un exit hace POST del ECB → el bucle despierta con WAIT	Aviso instantáneo y cero consultas en vano	APF + STC : trámite con sistemas

Plan: B primero (ZTKMONS) y C al final. B ya sirve por sí solo, obliga a entender lo que dice JES, y C reusa casi todo B: sólo cambia **quién despierta al bucle**. En B lo despierta el timer; en C, JES.

Fuente: IDEAS_cierres_produccion.md, IDEA-005 (2026-09-30)

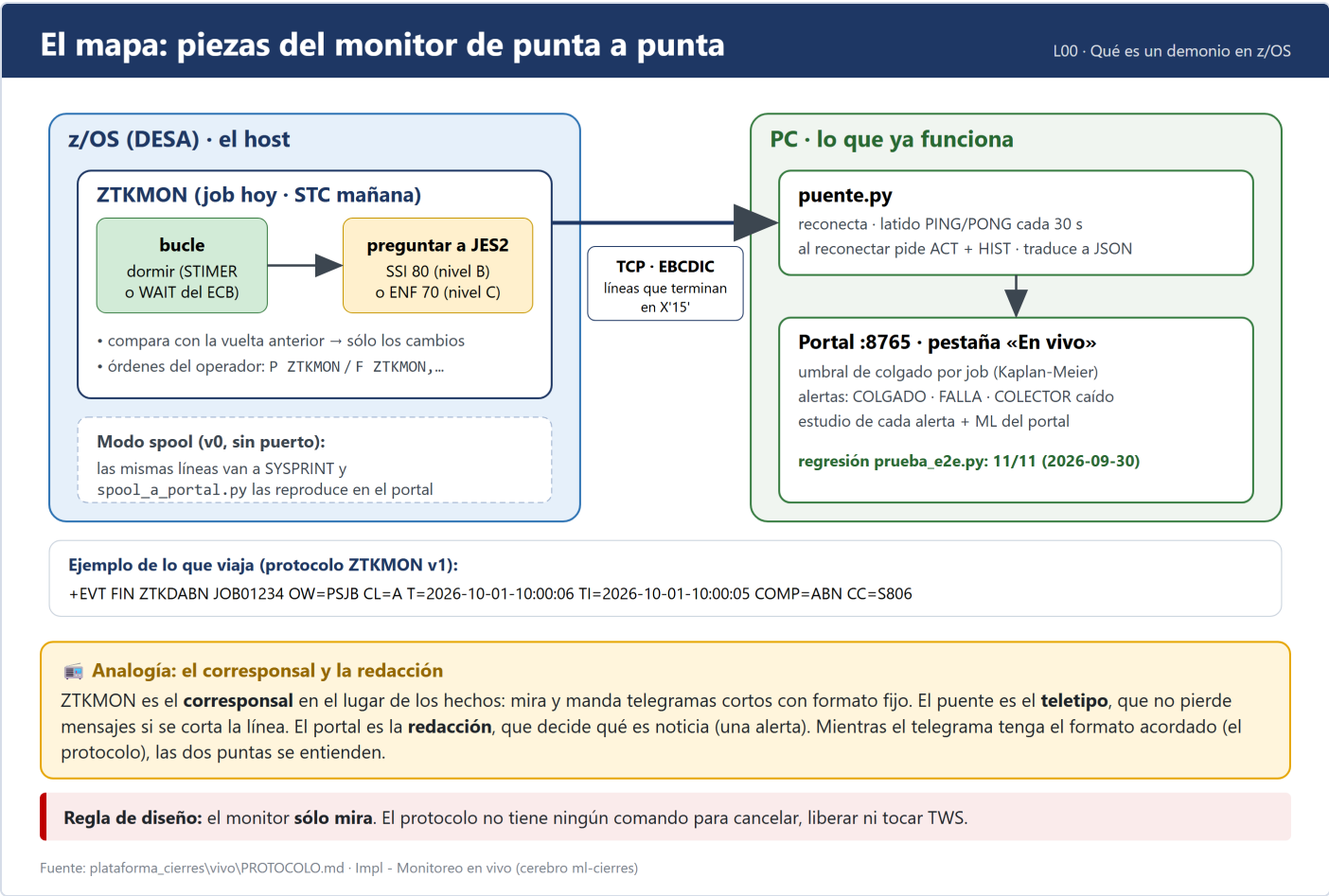
Lámina L00-05_tres_niveles

4. Las piezas del monitor



- El **contrato** entre los dos lados es `PROTOCOLO.md` (ZTKMON v1). Todo lo que funcione contra el simulador funciona contra el host si el host cumple el contrato.

- El monitor **sólo mira**: no existe ningún comando que cambie algo en el host.



1. ¿TIME DEC,...,DATATYPE=YYYYMMDD deja la fecha en los bytes 8-11?
2. ¿Un job que está ejecutando figura con STTRPHAZ = STAT_ONMAIN (14)?
3. ¿Un error de JCL aparece en STAT_POSTEX ?
4. ¿STATVRBO contesta desde un programa no autorizado y trae STVBXTST / STVBXTET ?
5. ¿Cuánta CPU gasta cada vuelta?

Dónde estamos (2026-10-05)

L00 · Qué es un demonio en z/OS

Pieza	Estado	Evidencia
Protocolo ZTKMON v1	✓ escrito	vivo\PROTOCOLO.md
Puente, simulador, pestaña «En vivo»	✓ andando	prueba_e2e.py 11/11 (2026-09-30)
ZTKMONS v0 (nivel B, modo spool, sin APF)	⚠ escrito, nunca ensamblado	vivo\host\ZTKMONS.asm , 944 líneas
Socket, STC, ENF 70, GQSCAN	🕒 por hacer	Módulos 6 a 9

Las 5 preguntas que contesta la primera práctica real (módulo 4)

#	Pregunta	Qué línea del monitor lo muestra
1	¿TIME DEC,...,DATATYPE=YYYYMMDD deja la fecha en los bytes 8-11?	La fecha del +HOLA
2	¿Un job ejecutando figura con STTRPHAZ = STAT_ONMAIN (14)?	Sí no, salen +DIAG FASE y ningún INI
3	¿Un error de JCL aparece en STAT_POSTEX ?	El +EVT FIN de ZTKDJCL
4	¿STATVRBO contesta sin APF y trae STVBXTST / STVBXTET ?	Los TI= de cada FIN
5	¿Cuánta CPU gasta cada vuelta?	JESMSG LG de ZTKMONR ÷ 240 vueltas

Fuente: vivo\host\LEEME.md · L00, sección 5

Lámina L00-08_donde_estamos

6. Preguntas de comprobación

Contestalas con tus palabras; no hace falta el detalle técnico todavía.

1. ¿Por qué un demonio que hace while(1){ preguntar(); } sin dormir es un mal demonio? ¿Qué recurso del sistema gasta?
2. ¿Qué diferencia práctica hay entre que ZTKMON sea un **job batch largo** y que sea un **STC**? Pensá en los iniciadores y en cómo lo para el operador.
3. En el nivel C, ¿quién despierta al bucle principal y con qué instrucción? ¿Y en el nivel B?
4. ¿Por qué el protocolo no tiene ningún comando para cancelar un job, aunque técnicamente se podría?

5. Si el puente de la PC se cae 10 minutos y un job arranca **y termina** en ese lapso, ¿cómo se entera el portal? (Pista: `PROTOCOLO.md`.)

Mis respuestas

Relacionado: [Ruta de aprendizaje](#) · [Glosario](#) · siguiente: [L01 - Anatomia de un programa Assembler - el prologo de ZTKMONS](#).

Fuente: IDEAS_cierres_produccion.md (IDEA-005, mediciones del 2026-09-30) · plataforma_cierres\vivo\PROTOCOLO.md · vivo\host\LEEME.md · manuales de IBM citados en la sección 2.

